

The solution:

Multi-agent

architecture

Split responsibilities across specialists:

```
Python
from google.adk.agents import LlmAgent

# Specialist 1: Billing expert
billing_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='billing_specialist',
    instruction="You are a billing expert. Handle payment questions."
)

# Specialist 2: Technical expert
technical_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='technical_specialist',
    instruction="You are a tech support expert. Help with technical issues."
)

# Coordinator routes to specialists
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='coordinator',
    instruction="Route billing questions to billing_specialist, tech questions to technical_specialist",
    sub_agents=[billing_agent, technical_agent]
)
```

What goes wrong:

- Specialized expertise - Each agent focuses on one domain
- Better answers - Focused instructions produce better results
- Easy to maintain - Add new specialists without changing existing agents
- Independent testing - Test each agent separately

Core concepts

1. Parent-child hierarchy with `sub_agents`

From ADK documentation

“In ADK, multi-agent systems are organized hierarchically. Agents can have parent-child relationships, where parent agents orchestrate or delegate to child agents (sub-agents). This is primarily established using the `sub_agents` parameter.”

How it works:

Python

```
from google.adk.agents import LlmAgent

# Child agents (specialists)
billing_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='billing_specialist',
    instruction="Handle billing questions"
)

technical_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='technical_specialist',
    instruction="Handle technical questions"
)

# Parent agent with sub_agents
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='coordinator',
    instruction="Coordinate specialists",
    sub_agents=[billing_agent, technical_agent]  # Creates parent-child
relationship
)
```

Reference: [ADK Docs - Agent Hierarchy](#)



Key points:

- `sub_agents` parameter: creates the parent-child relationship
- Parent coordinates: children specialize
- Single parent rule: each agent has at most one parent
- Navigation: `agent.parent_agent` returns the parent, `agent.find_agent(name)` finds children

The following diagram illustrates how a coordinator agent routes requests to specialized sub-agents. The coordinator receives all incoming requests and decides which specialist should handle each one based on the request content:

None

```
graph TB
    subgraph "Multi-Agent System"
        COORD[Coordinator] --> BILLING[Billing Specialist]
        COORD --> TECH[Technical Specialist]
    end

    USER[Customer Question] --> COORD
    BILLING --> ANSWER1[Billing Answer]
    TECH --> ANSWER2[Technical Answer]

    style COORD fill:#fff3cd
    style BILLING fill:#e1f5ff
    style TECH fill:#e1f5ff
```

The coordinator (yellow) receives requests and routes them to the appropriate specialist (blue). Each specialist handles its domain independently.

2. When to use multi-agent vs. single-agent

Use single agent when:

- Task fits one domain of expertise
- Instructions are simple and focused
- No need for specialized sub-tasks

Use multi-agent when:

- Task requires multiple domains of expertise
- Specialized instructions needed for different sub-tasks
- You want to test and maintain components independently

Reference: [ADK Docs - Multi-Agent Patterns](#)

Decision guide:

Factor	Single agent	Multi-agent
Scope	One domain	Multiple domains
Instructions	Short, focused	Would be long/complex
Maintenance	All in one place	Each specialist separate
Testing	Test one agent	Test each specialist

Rule of thumb: Start with a single agent. Move to multi-agent when your instruction becomes complex or you need multiple expertise areas.

Hands-on example

Building a customer service multi-agent system

What you’ll build: A coordinator that routes customer questions to billing and technical specialists.

Step 1: Create the project

Shell

```
adk create customer_service
cd customer_service
```



Step 2: Write the agent

Replace `agent.py` with:

```
Python
"""
Customer Service Multi-Agent System
A coordinator routes requests to billing and technical specialists.

Reference: https://google.github.io/adk-docs/agents/multi-agents
"""

from google.adk.agents import LlmAgent

# Specialist 1: Billing
billing_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='billing_specialist',
    description='Handles billing and payment questions',
    instruction="""
    You are a billing specialist.

    Help customers with:
    - Payment processing
    - Refunds
    - Billing inquiries
    - Invoice questions

    Be clear about policies and timelines.
    """
)

# Specialist 2: Technical Support
technical_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='technical_specialist',
    description='Handles technical issues and troubleshooting',
    instruction="""
    You are a technical support specialist.

    Help customers with:
    - Login problems
    - App crashes
    - Feature troubleshooting
    - Configuration help

    Ask clarifying questions to diagnose issues.
    """
)

# Coordinator
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='customer_service_coordinator',
    description='Routes customer requests to specialists',
    instruction="""
    You are a customer service coordinator.

    Route requests to the right specialist:
    - Billing/payment issues → billing_specialist
    - Technical problems → technical_specialist
    - Simple general questions → answer directly

    Be friendly and helpful.
    """,
    sub_agents=[billing_agent, technical_agent]
)
```

What this code does:

- Two specialists with focused instructions (billing and technical)
- One coordinator that routes requests based on the topic
- `sub_agents` parameter creates the parent-child relationships

Step 3: Run and test

```
Shell  
adk web
```

Navigate to `http://localhost:8000`.

Step 4: Test scenarios

Test 1: Simple calculation

You: `I was charged twice for my subscription this month`

Expected behavior:

- Coordinator routes to `billing_specialist`, who handles the payment issue.

Test 2: Technical question

You: `The app crashes when I try to upload a file`

Expected behavior:

- Coordinator routes to `technical_specialist`, who helps troubleshoot.

Test 3: General question

You: `What are your business hours?`

Expected behavior:

- Coordinator answers directly without routing to a specialist.

Key takeaways

This part introduced the foundational concepts of multi-agent systems in ADK. Here are the key points to remember as you move forward:

What is multi-agent:

- Multiple specialized agents working together
- Each agent focuses on one domain
- Parent coordinates, children specialize

When to use multi-agent:

- Multiple domains of expertise needed
- Instructions would be complex for one agent
- Want independent testing and maintenance

How to build multi-agent:

- Create specialist agents with focused instructions
- Create coordinator with `sub_agents` parameter
- Coordinator routes requests to appropriate specialists

Remember: Start simple with a single agent. Move to multi-agent when your use case naturally requires multiple expertise areas.

